

Technical Research: Evaluation of LLM Observability & Drift-Detection Platforms

Purpose

Before presenting this project as a finished result, it's worth being clear about where it sits in the existing landscape. This page surveys the commercial and open-source LLM observability space, plus the closest academic parallel, and states plainly what's genuinely novel here versus what's a validated implementation of an established idea.

Scope Note: A Different Category Than Real-Time Firewalls

This research covers **observability and drift-detection platforms** — tools for understanding production LLM behavior over time. It's a different category from the real-time inline guardrails and prompt-injection firewalls (Lakera, Prompt Security, LLM Guard, Cisco AI Defense) already researched separately for this epic. Both categories matter; they solve different problems. A real-time firewall decides whether to block a request *right now*, using pattern matching or a lightweight classifier. An observability platform asks a slower, broader question: *across all recent traffic, does something look wrong* — which is what this project builds toward.

The Commercial and Open-Source Landscape

Platform	Core approach	Relevant to this project
Galileo	Managed platform; purpose-built evaluation models (Luna) analyze production traffic for failure modes, edge cases, and drift	Closest commercial parallel — automated behavioral pattern analysis over live traffic
WhyLabs (LangKit)	Profile-based monitoring: summarizes data locally via why logs, tracks distribution shift using statistical drift algorithms (Hellinger, KL Divergence, Jensen-Shannon, PSI)	Statistically closest parallel — distribution-shift detection is conceptually the same problem this project solves, though WhyLabs works on summarized profiles rather than full embeddings
Confident AI / DeepEval	Open-source evaluation framework; alerts on evaluation-score regressions including drift on a specific prompt or use case; integrates into CI via <code>pytest</code>	Strong parallel on the "automated, repeatable validation" side of this project
Arize Phoenix	Open-source, OpenTelemetry-native; accepts traces via standard OTLP protocol	Different architecture (trace-based, not embedding-similarity-based), but same general observability goal
Datadog LLM Observability	Infrastructure-native; extends classical APM into LLM-specific signals	Relevant primarily as the "already paying for it" option for teams already on Datadog — not embedding/drift-focused in the way this project is
LangSmith / Langfuse	Tracing and evaluation platforms, deep framework integration (LangChain /LangGraph) or framework-agnostic	General LLM observability; less focused on semantic drift specifically

None of these are purpose-built, gateway-native security tools for prompt-injection detection specifically — they're general-purpose LLM observability platforms that *can* be pointed at this problem, not tools designed around it from the start. That's a meaningful gap this project's architecture fits into.

The Closest Academic Parallel

"Are you still on track!? Catching LLM Task Drift with Activations" (Abdelnabi, Fay, Cherubin, Salem, Fritz, Pavard — Microsoft Security Response Center and CISPA Helmholtz Center, 2024, [arXiv:2406.00799](https://arxiv.org/abs/2406.00799)) is the nearest research parallel to this project's core idea, and worth knowing precisely how it differs, not just that it exists.

Their method — TaskTracker — detects prompt injection by comparing an LLM's **internal activations** before and after processing external input, using a linear classifier on the resulting "activation delta." Reported results are strong: near-perfect ROC AUC on out-of-distribution test data, generalizing to unseen attack types without being trained on them.

The key architectural difference: TaskTracker requires access to the model's internal activations — it has to run inside or alongside the model itself. This project instead uses **external text embeddings** of captured prompts and completions, computed independently of the model that generated them. That's a deliberate tradeoff: TaskTracker's approach is likely more sensitive (it sees the model's actual internal state), but it only works when you control or can instrument the model directly. An embedding-based, gateway-level approach works regardless of which model sits behind the gateway — including third-party or black-box models where activation access isn't possible at all. Given this project's context (a gateway sitting in front of traffic to models it doesn't necessarily control), the embedding-based approach is the only one of the two that's actually deployable here.

Where This Project Sits

Stated plainly: the core idea — semantic, continuous drift detection over LLM traffic — is not new. It's an active area with real commercial products and peer-reviewed research behind it. This project's contribution isn't the concept; it's a working, validated, gateway-integrated implementation of that concept, with two things worth calling out specifically:

- **A measured, reproducible finding** — the jump from 41% to 100% drift recall by moving from a single baseline centroid to a multi-cluster model — validated against ground-truth labels, not asserted.
- **Production-hardening work** most PoC-stage implementations skip — index correctness under continuous ingestion, migration safety under concurrent replicas — the kind of detail that separates a demo that works once from a system that would survive actually running.